

Verilog 第 3 次实验报告

学生姓名	王一诺	学号	2312331	指导老师	董前琨老师
------	-----	----	---------	------	-------

一、实验内容及步骤

1. 完成 nandgame 里面的 Switching 和 Arithmetic Logic Unit:

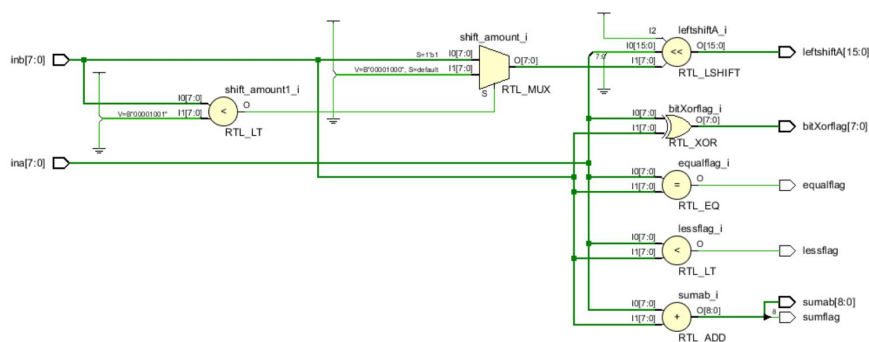
大概用时 40 分钟左右，逻辑算术单元部分读懂题目理解要求反而是比较耗时的地方，读懂之后设计逻辑就比较顺手，由此可见设计逻辑电路时，读懂、掌握题目要求是重点，构建好输入和输出的关系之后一切都迎刃而解。在这一部分中大量使用了选择器，可见构建好后的单元部件要熟知其功能并加以灵活运用。

2. 自行设计一个简单运算模块：

代码以及简要思路：

```

module simple_operations(
    input [7:0] ina,      // 8位输入ina
    input [7:0] inb,      // 8位输入inb
    output [8:0] sumab,    // 9位输出，用于存储ina和inb的和以及进位
    output sumflag,        // 输出，表示是否产生进位
    output [15:0] leftshiftA, // 16位输出，用于存储ina左移inb位的结果
    output lessflag,       // 输出，表示ina是否小于inb
    output equalflag,      // 输出，表示ina是否等于inb
    output [7:0] bitXorflag // 8位输出，用于存储ina和inb按位异或的结果
);
// Calculate sum and carry
assign sumab = ina + inb;      // 将ina和inb相加，结果存储在9位宽的sumab中
assign sumflag = sumab[8];     // 进位标志是sumab的第9位
// Left shift ina by inb positions
wire [7:0] shift_amount = inb < 9 ? inb : 8; // 限制左移位数不超过8位
assign leftshiftA = ina << shift_amount;     // 将ina左移shift_amount位
// Less than flag
assign lessflag = (ina < inb) ? 1'b1 : 1'b0; // 如果ina小于inb，则lessflag为1，否则为0
// Equal flag
assign equalflag = (ina == inb) ? 1'b1 : 1'b0; // 如果ina等于inb，则equalflag为1，否则为0
// Bitwise XOR
assign bitXorflag = ina ^ inb; // 将ina和inb按位异或，结果存储在bitXorflag中
endmodule
    
```



输出示例：

Name	Value	107, 172 ps	107, 173 ps	Name	Value	121, 614 ps	121, 615 ps	121, 616 ps
ina[7:0]	aa		aa	ina[7:0]	80			80
inb[7:0]	cc		cc	inb[7:0]	03			03
sumab[8:0]	176		176	sumab[8:0]	083			083
sumflag	1			sumflag	0			
leftshiftA[15:0]	aa00		aa00	leftshiftA[15:0]	0400			400
lessflag	1			lessflag	0			
equalflag	0			equalflag	0			
bitXorflag[7:0]	66		66	bitXorflag[7:0]	83			83

二、总结

1. 做逻辑设计的关键是读懂题目要求，想清楚输入和输出的关系
2. 运用了一下双目运算符，感觉真的和 C++ 有异曲同工之妙
3. 还是写 tb 文件有点不熟练，要熟悉一下随时间更改输入信号的写法（尽管已经写了两次了有时还是会写错）
4. 这一次完成 nandgame 耗时更久了些，除了 1 所提到的读题困难，此外就是处理条件选择时的思路。要大胆设想，捋清思路（这里就是思考怎么把二选一扩展到四选一），连线的时候就会丝滑一些。